

Extending GPT-2 for Informal and Slang Aware Language Understanding

Stanford CS224N {Default}

Ruby Hernandez
Department of Computer Science
Stanford University
rubyh@stanford.edu

Dhruv Naik
Department of Computer Science
Stanford University
naikdhr@stanford.edu

Abstract

Pretrained language models must often be adapted to handle diverse linguistic styles, including informal slang and structured poetic language. In this project, we investigate how a pretrained GPT-2 model can be fine-tuned to perform both classification and generation tasks within the CS224N default project framework. While we first evaluate GPT-2 on sentiment classification as a baseline task, our primary focus is on paraphrase detection, Shakespearean sonnet generation, and an extension that models informal Gen-Z slang.

For paraphrase detection, we implement a cloze-style prompting formulation using GPT-2 to determine whether two questions express the same meaning. For sonnet generation, we fine-tune GPT-2 as an autoregressive language model to generate poetic continuations conditioned on the first three lines of a Shakespeare sonnet. We further extend the framework to model informal language by generating slang paraphrases and detecting valid slang translations.

Our results show that full-model fine tuning improves sentiment classification performance, and that prompt-based paraphrase detection achieves strong accuracy (0.895 dev, 0.881 test). The sonnet generation model achieves a chrF score of 41.9, indicating that GPT-2 captures stylistic patterns of Shakespearean language. For slang detection, our fine-tuned GPT-2 classifier achieves 0.965 development accuracy compared to a 0.500 zero-shot baseline, demonstrating that task-specific fine-tuning substantially improves informal language understanding.

1 Key Information to include

- Mentor: Wei Liu
- External Collaborators (if you have any): N0
- Sharing project: No
- Late Days (if you are using any):

Team member	Late days remaining
Ruby Hernandez	5
Dhruv Naik	5
Total pooled	10
Team size	2
Late days applied for the report	1

2 Introduction

Natural language models are increasingly deployed in settings that demand not only grammatical fluency but also alignment with how people actually speak. In many online communities and everyday

conversations, this means navigating a rich space of slang, internet vernacular, and rapidly evolving “Gen-Z” expressions. Because slang often carries pragmatic cues about stance, in-group membership, and emotional tone that are not captured by standard formal text, models that ignore it risk sounding stiff, outdated, or misinterpreting user intent. Developing systems that can *understand*, *detect*, and *generate* slang is therefore an important step toward more human-like language modeling: it enables assistants to better interpret informal inputs, flag or reinterpret ambiguous slang usage, and respond in a register that matches the user’s style. In this project, we study how a pretrained GPT-2 model can be adapted to multiple tasks through fine-tuning within the CS224N default project framework. We evaluate GPT-2 on both classification and generative tasks, focusing on paraphrase detection and Shakespearean sonnet generation. In addition, we explore an extension that models informal language by generating and detecting Gen-Z slang expressions. Together, these experiments provide insight into how transformer-based language models adapt to different forms of language understanding and generation.

3 Related Work

Large pretrained language models have become a foundation of modern natural language processing systems. Radford et al. [1] introduced GPT-2, demonstrating that large transformer based language models trained on web scale corpora can perform a wide range of language tasks through autoregressive next-token prediction. Their results show that such models capture rich linguistic representations and are capable of generating coherent long form text, making them suitable for both language understanding and generation tasks.

Recent work has also examined how large language models handle informal language and slang. Sun et al. [2] study how modern language models represent and process slang expressions, highlighting that informal language often carries significant cultural and social meaning rather than simply acting as noise. Their findings suggest that language models may struggle with informal linguistic variation, motivating further research on adapting pretrained models to different language styles.

Our work builds on these directions by fine-tuning GPT-2 across both classification and generative tasks, including paraphrase detection and Shakespearean sonnet generation, while additionally exploring how fine-tuning can improve performance on informal language through Gen-Z slang detection and generation.

4 Approach

Our project builds on the GPT-2 architecture provided in the CS224N default project and adapts it to three downstream tasks: sentiment classification, paraphrase detection, and sonnet generation. While the base transformer implementation follows the provided starter code, we design task-specific training formulations and decoding strategies to evaluate the model’s ability to perform both classification and generative language tasks. In all experiments we fine-tune a pretrained GPT-2 model using the AdamW optimizer. In addition, we explore an extension that translates standard English into Gen-Z slang using a transformer-based translation model.

4.1 Baselines

As baselines, we use the GPT-2 models provided in the CS224N default project starter code. The baseline system consists of a pretrained GPT-2 transformer used as a contextual encoder, with task-specific training procedures defined in the project framework.

For sentiment classification, the starter implementation evaluates two fine-tuning regimes: a *last-linear-layer* setting in which GPT-2 parameters are frozen and only the classification layer is trained, and a *full-model* setting in which all GPT-2 parameters are updated during training. Using the provided implementation, these baselines achieve development accuracies of 0.463 and 0.518 on SST, and 0.861 and 0.976 on CFIMDB for the last-linear-layer and full-model settings, respectively.

For paraphrase detection, the project handout provides a baseline cloze-style prompting formulation using GPT-2. The baseline system achieves a development accuracy of 0.882 and a test accuracy of 0.856 on the Quora Question Pairs dataset.

For sonnet generation, the baseline GPT-2 language model produces continuations conditioned on the first three lines of a sonnet. The baseline achieves chrF scores of 41.414 on the development set and 41.208 on the test set. These baseline results provide reference points for evaluating the improvements achieved by our fine-tuned models across the tasks.

Finally, for slang detection, we additionally evaluate a zero-shot GPT-2 baseline that attempts to classify slang usage without task-specific fine-tuning. This baseline achieves an accuracy of 0.500, which serves as a reference point for evaluating the improvements obtained through fine-tuning on the slang detection dataset.

4.2 Sentiment Classification

We first evaluate our implementation of GPT-2 on sentiment classification using the SST and CFIMDB datasets. Following the default project framework, GPT-2 is used as a contextual encoder and the representation of the final token is treated as a sentence-level embedding. This representation is passed through a dropout layer and a linear projection to produce logits over sentiment labels.

We evaluate two fine-tuning regimes. In the last-linear-layer setting, the parameters of GPT-2 are frozen and only the classification layer is trained. In the full-model setting, all GPT-2 parameters are updated during training. Comparing these two settings allows us to measure how much task-specific adaptation improves performance beyond the pretrained representations.

4.3 Paraphrase Detection

For paraphrase detection we fine-tune GPT-2 on the Quora Question Pairs dataset using the cloze-style formulation suggested in the project handout. Each sentence pair is converted into a natural language prompt asking whether the two questions are paraphrases.

The prompt is tokenized and processed by GPT-2 to produce contextual hidden states across the sequence. We use the hidden representation corresponding to the final token as the representation of the sentence pair. Let $h_{\text{last}} \in \mathbb{R}^d$ denote this final hidden state returned by GPT-2.

Instead of introducing an additional classifier head, we reuse GPT-2’s language modeling output layer to map this representation back into the vocabulary space:

$$z = \text{LMHead}(h_{\text{last}}) \quad (1)$$

This produces logits over the entire vocabulary. The predicted distribution is then computed using

$$\hat{y} = \text{softmax}(z) \quad (2)$$

During training we optimize the model using cross-entropy loss

$$\mathcal{L}_{\text{para}} = - \sum_c y_c \log \hat{y}_c \quad (3)$$

so that the model learns to predict the correct label token “yes” (token id 8505) or “no” (token id 3919) at the end of the prompt. Although the input is phrased as a cloze-style question, training effectively treats the problem as supervised paraphrase classification using the final token representation.

Training uses the AdamW optimizer with weight decay, and we apply early stopping based on development accuracy to prevent overfitting.

4.4 Sonnet Generation

To evaluate the generative capabilities of GPT-2, we fine-tune the model on a corpus of Shakespearean sonnets. Unlike the paraphrase task, generation requires predictions at every token position in the sequence. Given a tokenized sonnet, GPT-2 produces contextual hidden states for each position, which are then mapped back into the vocabulary space using the tied output projection layer.

The model is trained as an autoregressive language model that predicts the next token in the sequence. Concretely, the logits are shifted so that the representation at position t predicts token x_{t+1} . The training objective minimizes cross-entropy loss over all non-padding tokens in the sequence:

$$\mathcal{L}_{\text{sonnet}} = - \sum_{t=1}^{T-1} \log P(x_{t+1} \mid x_{\leq t}) \quad (4)$$

Padding positions are masked from the loss to avoid affecting the training signal. Because the sonnet dataset is relatively small, we track training loss across epochs, save the best checkpoint, and apply early stopping to reduce overfitting. We also apply gradient clipping to stabilize optimization during fine-tuning.

At generation time, the model produces sonnet continuations autoregressively starting from a prompt consisting of the first three lines of a sonnet. To balance coherence and diversity in the generated text, we combine temperature scaling with both top- k and nucleus (*top-p*) sampling. In top- k filtering, sampling is restricted to the k tokens with the highest probabilities:

$$P_k(x_i) = \begin{cases} \frac{P(x_i)}{\sum_{j \in K} P(x_j)} & \text{if } i \in K \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

where K denotes the set of the top- k tokens ranked by probability. The remaining tokens are renormalized to form a valid distribution before sampling. We additionally apply a light repetition penalty to discourage repeated tokens and introduce a bias toward newline tokens once a generated line becomes long, helping the model better follow the structure of Shakespearean sonnets.

4.5 Slang Extension

In addition to the original sonnet generation and paraphrase detection tasks, we implemented a pair of Gen-Z slang extensions that closely mirror these setups while exploring different modeling choices. For slang generation, we fine-tune a Hugging Face GPT2LMHeadModel on the Programmer-RD-AI/genz-slang-pairs-1k dataset, treating the task as conditional language modeling. Given a formal phrase x and its slang target y , we construct a prompt

$$p(x) = \text{"Translate to Gen-Z slang: " } x \text{"\nSlang: "}$$

and train the model to maximize the conditional likelihood $p_\theta(y \mid p(x))$. Concretely, we minimize the token-level cross-entropy

$$\mathcal{L}_{\text{gen}}(\theta) = - \sum_{t \in \mathcal{Y}} \log p_\theta(y_t \mid p(x), y_{<t}), \quad (6)$$

where $\mathcal{Y}$ indexes only the slang continuation (prompt tokens are masked with label -100). At inference time, we decode using a mixture of top- k and nucleus (top- p) sampling with repetition penalties and a heuristic scoring function that favors fluent, non-repetitive, and mostly ASCII outputs. We then compare generations from the base GPT-2 and the fine-tuned model side-by-side on a small set of held-out formals, and also provide an interactive mode where the user can enter arbitrary formal sentences and see both outputs. Qualitatively, the fine-tuned model produces shorter, more on-topic, and more colloquial slang paraphrases, whereas the base GPT-2 tends to stay closer to the original phrasing or drift into generic completions.

For slang detection, we adapt the paraphrase detection setup to a binary classification problem over (formal, candidate slang) pairs. From the same Gen-Z dataset we build a labeled set where positive examples are gold (formal, slang) pairs and negative examples are created by sampling mismatched slang for each formal while avoiding obvious collisions. Each example is converted into a cloze-style prompt:

$$q(x, s) =$$

“Formal: “ x “\nCandidate slang: “ s “\nIs the candidate a correct Gen-Z slang translation? Answer “yes” or “no”:

which is encoded by the in-repo GPT-2 backbone to produce the last-token hidden state $h_{\text{last}} \in \mathbb{R}^d$. We train a linear head $W \in \mathbb{R}^{2 \times d}$, $b \in \mathbb{R}^2$ with logits

$$z = Wh_{\text{last}} + b, \quad p_{\theta}(y | x, s) = \text{softmax}(z), \quad (7)$$

and minimize the binary cross-entropy

$$\mathcal{L}_{\text{det}}(\theta) = -[y \log p_{\theta}(1 | x, s) + (1 - y) \log p_{\theta}(0 | x, s)], \quad (8)$$

where $y \in \{0, 1\}$ indicates whether the candidate slang is correct. As a zero-shot baseline, we reuse the same GPT-2 backbone without the classifier head, project h_{last} to vocabulary logits, and compare the scores of the vocabulary tokens corresponding to “yes” and “no”; the higher logit determines the prediction. Early experiments show that the fine-tuned classifier substantially improves dev accuracy over this zero-shot baseline, and we also export per-example predictions so its behavior can be inspected and compared to the generation model’s outputs.

5 Experiments

5.1 Data

We evaluate our models on datasets corresponding to the three tasks explored in this project. For sentiment classification, we use the SST and CFIMDB datasets provided in the CS224N default project.

For paraphrase detection, we use the Quora Question Pairs dataset, which contains pairs of questions labeled according to whether they express the same semantic meaning.

For sonnet generation, we use the Shakespeare sonnet corpus provided in the project starter code. During training, the model learns to predict the next token in the sonnet sequence, and during evaluation it generates the continuation of a sonnet conditioned on the first three lines.

For our slang translation extension, we use a HuggingFace dataset [3] containing paired examples of standard English sentences and their corresponding Gen-Z slang expressions.

5.2 Evaluation method

We evaluate model performance using task-specific metrics. For sentiment classification and paraphrase detection, we report accuracy on the development and test sets. Accuracy measures the proportion of predictions that match the ground truth labels.

For sonnet generation, we evaluate generated text using the chrF metric, which measures character-level n-gram overlap between generated text and reference sonnets. This metric is commonly used for evaluating generative language models and is the official metric used by the project leaderboard.

5.3 Experimental details

All experiments fine-tune a pretrained GPT-2 model using the AdamW optimizer. We initially train models with a learning rate of 1×10^{-3} and later reduce the learning rate to 1×10^{-5} for additional fine-tuning to improve task performance.

For sentiment classification we compare two training settings: last-linear-layer fine-tuning, where GPT-2 parameters are frozen and only the classifier head is trained, and full-model fine-tuning, where all GPT-2 parameters are updated.

For paraphrase detection we fine-tune the full GPT-2 model using the cloze-style formulation described in Section 3.2. Training includes weight decay regularization and early stopping based on development set performance.

For sonnet generation we fine-tune GPT-2 as an autoregressive language model using next-token cross-entropy loss. Gradient clipping is applied during training to stabilize optimization. During inference, text is generated autoregressively using temperature scaling together with top-k and top-p sampling to balance coherence and diversity in the generated poems.

For slang detection we fine-tune the full GPT-2 encoder with a lightweight two-layer classification head on top of the final token representation of a (formal, candidate slang) prompt. Training uses

the AdamW optimizer with a learning rate of $1 \cdot 10^{-5}$, batch size 8, balanced positive/negative examples generated by hard negative sampling from the same slang pool, and model selection based on development accuracy.

For slang generation we fine-tune GPT-2 as a conditional language model on paired (formal, slang) examples using the Hugging Face Trainer with cosine learning-rate decay, weight decay of 0.01, small batch size (4) with gradient accumulation (effective batch size 16), and early stopping on validation loss. At inference time we decode slang continuations with constrained beam search and nucleus-style sampling (temperature and top-p), together with simple fluency/ASCII heuristics and a fallback template, to encourage coherent yet diverse slang paraphrases that remain readable.

5.4 Results

Table 1 summarizes the quantitative performance of our models across all tasks and compares them to the baseline settings provided in the project framework.

For sentiment classification, full-model fine-tuning improves performance on SST, achieving a development accuracy of 0.518 compared to 0.463 when only the final classifier layer is trained. On CFIMDB, both training settings perform strongly, with the full-model configuration achieving an accuracy of 0.976.

Our sentiment classification results are consistent with expectations for transformer fine-tuning. Updating the full GPT-2 model improves performance on SST compared to training only the final classification layer, suggesting that adapting the pretrained representations to the task is beneficial for more nuanced sentiment prediction. In contrast, performance on CFIMDB is already strong with the frozen encoder, indicating that the pretrained GPT-2 representations capture much of the sentiment information required for the binary classification task.

For paraphrase detection, our cloze-style prompting approach achieves a development accuracy of 0.895 and a test accuracy of 0.881. These results indicate that prompting GPT-2 with natural language questions is an effective strategy for semantic similarity tasks.

For sonnet generation, our model achieves chrF scores of 42.601 on the development set and 41.905 on the test set. Our chrF scores indicate that the generated sonnets capture character-level patterns present in Shakespeare’s writing, while the sampling strategy helps balance coherence and diversity in the generated outputs. The results demonstrate that GPT-2 can learn stylistic patterns from a relatively small corpus of poetic text.

Task	Metric	Dev	Test
SST (Last Linear Layer)	Accuracy	0.463	–
SST (Full Model)	Accuracy	0.518	–
CFIMDB (Last Linear Layer)	Accuracy	0.861	–
CFIMDB (Full Model)	Accuracy	0.976	–
Paraphrase Detection	Accuracy	0.895	0.881
Sonnet Generation	chrF	42.601	41.905

Table 1: Performance of our GPT-2 models across all tasks.

For our slang modeling extension, Table 2 compares a zero-shot GPT-2 baseline with our fine-tuned slang detection model. Fine-tuning substantially improves performance, increasing accuracy from 0.500 to 0.965 on the development set. This large improvement suggests that pretrained language models benefit significantly from task-specific supervision when adapting to informal language.

Task	Metric	Dev
Slang Detection (Zero-shot GPT-2)	Accuracy	0.500
Slang Detection (Fine-tuned GPT-2)	Accuracy	0.965

Table 2: Comparison of slang detection performance between a zero-shot GPT-2 baseline and our fine-tuned GPT-2 classifier.

6 Analysis

6.1 Sonnet Generation

We qualitatively evaluate our fine-tuned GPT-2 model by inspecting generated sonnet continuations. The model is given the first three lines of a Shakespeare sonnet and must generate the remaining lines autoregressively. Table 3 shows an example continuation produced by our model. While chrF provides a quantitative evaluation of character-level similarity, qualitative inspection is necessary to understand whether the generated poems capture stylistic features of Shakespearean writing.

Prompt (first three lines)

My love is as a fever, longing still
For that which longer nurseth the disease,
Feeding on that which doth preserve the ill,

Generated continuation

Like an autumnal infection so dear to our picture. Yet thence I find no plague;
The birds sing and grow merry in me like summer’s green,
But my breast heals more easily after three winters long. Then with April going fair we
walk alone
In sweet spite of thorns growing everywhere. But then fear did dispense him joy:
Thy hand should cleanse his veins where he died sick
And thou wilt live replete by thy side! And yet peace prevailed—
Because trees loved never die nor fall dead either,
Nor they but buds do play hymns for thee at night.

Table 3: Example sonnet continuation generated by our fine-tuned GPT-2 model.

The generated continuation captures several stylistic features of Shakespearean poetry, including archaic vocabulary (e.g., “thou”, “thy”) and imagery related to nature and seasons such as “summer’s green” and “three winters long”. These patterns suggest that the model learns stylistic and thematic elements from the training corpus.

However, the model does not consistently maintain the global structure of a sonnet. Some lines become overly long, topics shift abruptly, and the generated text does not reliably follow Shakespearean rhyme or meter. This likely results from the small size of the sonnet dataset and the next-token prediction objective, which does not explicitly enforce poetic structure.

Overall, our GPT-2 model captures surface-level stylistic patterns of Shakespearean language but struggles with deeper structural constraints of sonnet form.

6.2 Slang Extension

We qualitatively evaluate the slang generation model by comparing outputs from the base GPT-2 language model with those from our fine-tuned variant on a set of informal prompts. The model is trained with a conditional language modeling objective, maximizing $\sum_t \log p_\theta(w_t \mid w_{<t}, \text{“Translate to Gen-Z slang: ”}, x)$, where x is the formal input sentence and loss is applied only on the slang continuation. Training loss decreases from roughly $\mathcal{L}_{\text{train}} \approx 3.4$ to ≈ 1.0 , and the final dev loss stabilizes near 1.10, indicating that the model learns a more specialized slang distribution while remaining imperfect. To illustrate the effect of this fine-tuning, Table 4 shows a few representative prompts with both base and fine-tuned outputs.

The base model tends to produce generic, expository continuations that are only loosely related to the input sentiment (e.g., for “I am extremely tired” it drifts into a description of a “popular podcast”), reflecting the broad web distribution it was originally trained on. In contrast, the fine-tuned model reliably injects slang markers and affective language (e.g., “sooo mad”, “vibing with it”, “no cap”), and more directly mirrors the polarity of the prompt (positive for “good day”, negative for “extremely tired”). At the same time, the setup has clear limitations: the small and stylistically narrow slang dataset leads to occasional artifacts (e.g., odd fragments or truncated phrases), and the next-token objective with heuristic decoding does not explicitly control for grammaticality or diversity, so some outputs are syntactically noisy or semantically shallow. Overall, fine-tuning shifts the distribution

Prompt	Base GPT-2 output	Fine-tuned GPT-2 output
That was really good	“that person is just great. And I can’t do anything about it because of my parents, but that’s the ...”	“That was just sooo mad, I can’t stop vibing with it”
I am extremely tired	“This was used on the popular podcast for many years by both fans and critics. If you have ever listened too long ...”	“I’m takin’ all the heat, I’m totally drained today”
I am having a good day	“a nice conversation, the guy who is just playing around with his phone when he should be watching tv or checking Facebook”	“I’m vibing mad today, no cap”

Table 4: Example Gen-Z slang generations from the base GPT-2 model and the fine-tuned slang generator.

toward recognizably Gen-Z slang, but the model still inherits biases of the base GPT-2 corpus and the constraints of a relatively small, noisy supervision signal.

7 Conclusion

In this project, we explored how a pretrained GPT-2 model can be adapted to multiple natural language understanding and generation tasks through fine-tuning. Within the CS224N default project framework, we evaluated GPT-2 on sentiment classification, paraphrase detection, and Shakespearean sonnet generation, while also extending the system to model informal language through Gen-Z slang detection and generation.

Our experiments show that task specific fine tuning significantly improves performance across tasks. Full model fine-tuning improves sentiment classification performance on SST, and the cloze-style prompting formulation achieves strong results for paraphrase detection with a development accuracy of 0.895 and a test accuracy of 0.881. For generative tasks, the fine tuned sonnet model achieves a chrF score of 41.9, indicating that GPT-2 captures stylistic patterns of Shakespearean language. In our slang extension, fine-tuning improves slang detection accuracy from a 0.500 zero-shot baseline to 0.965, demonstrating that pretrained language models benefit substantially from supervision when adapting to informal language styles.

Overall, our results suggest that transformer-based language models can adapt to both structured literary language and rapidly evolving informal language through targeted fine tuning. However, limitations remain: the sonnet model struggles with global poetic structure, and the slang generation model is constrained by the relatively small size and stylistic diversity of the training dataset. Future work could explore larger slang datasets, structural constraints for poetry generation, and stronger decoding strategies to improve coherence and linguistic diversity.

Team contributions (Required for multi-person team)

Ruby Hernandez - Primarily implemented the baseline GPT-2 training infrastructure, improved the SST and CFIMDB performance, helped with paraphrase detection, and did all of the sonnet generation models, including training pipelines and debugging. Contributed to coding and debugging across all project files and shared responsibility for writing the final report and preparing the project poster.

Dhruv Naik - Implemented the Adam optimizer, SST and CFIMDB section, implemented paraphrase detection functionality, and worked on all slang extension model training and code development. Contributed to coding and debugging across all project files and shared responsibility for writing the final report and preparing the project poster.

References

- [1] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.
- [2] Zhewei Sun, Qian Hu, Rahul Gupta, Richard Zemel, and Yang Xu. Toward informal language processing: Knowledge of slang in large language models. <https://arxiv.org/abs/2404.02323>, 2024.
- [3] Ranuga Disansa Gamage. genz-slang-pairs-1k (revision 9728f69). <https://huggingface.co/datasets/Programmer-RD-AI/genz-slang-pairs-1k>, 2025.